

RISC OS on a Pi 4 in QEMU - the 32-bit A72 changes

RISC OS on a Pi 4, in QEMU

How stock QEMU 11.1.0 was taught to boot the stock RISC OS 5.30 Raspberry Pi ROM, on an emulated Raspberry Pi 4 whose Cortex-A72 runs in 32-bit mode — and why almost none of the work was about the CPU.

The fork runs `-M raspi4b -cpu cortex-a72,aarch64=off` and reaches a networked desktop from the RISC OS Open SD card image, with a USB keyboard, a tablet pointer and Ethernet. The first desktop arrived on the first day.

The finding that shapes everything in this document:

The CPU half was free. The machine half was the work.

Stock QEMU already runs a Cortex-A72 in AArch32 on its `raspi4b` machine, given the right option. Every blocker was somewhere else. Each was a Linux-shaped assumption in QEMU's Raspberry Pi peripheral models, meeting an operating system that boots a Pi very differently from Linux.

These documents

Chapter	What it covers
1. The premise	why a 32-bit A72, and why the CPU was free
2. First light	a silent mailbox, and cores running garbage
3. Tracing beats theorising	the method, and five blockers to a prompt
4. QEMU is the firmware	CMOS, a crashing driver, the card, EDID

These documents, continued

Chapter	What it covers
5. Input and network through a FIQ	USB from the fast interrupt; CDC network
6. Time for an interrupt-driven OS	a timer thread, latches, DMA copies
7. Two hosts, and what is left	Macs, cores, upstreaming, open issues

At a glance

Base	QEMU 11.1.0 (84f07211cc), plus 156 commits to e885a1a429
Command line	-M raspi4b -cpu cortex-a72,aarch64=off
Guest	the stock RISC OS Open 5.30 Pi ROM
First desktop	day one: 18:14 from ROM, 19:57 from card
Boot	~27 s i7-12700, 20.8 s M4, 0.68 s snapshot
Hosts	Windows (Direct3D 11); macOS, Arm and Intel (Metal)

The published branch has since grown to 193 commits; the extra 37 are almost all HostFS, which has [its own walkthrough](#).

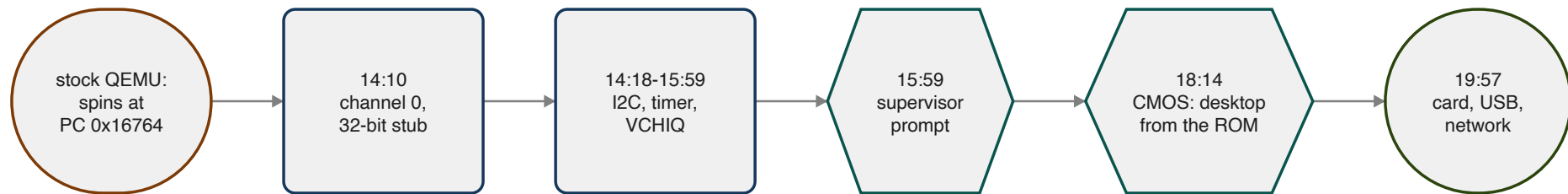
The shortest possible summary

QEMU's Raspberry Pi 4 model was written for Linux. Linux boots a Pi in AArch64, reads a device tree, powers devices through a modern property tag, and never touches half of the SoC's legacy interfaces. RISC OS 5 differs on almost every count. It runs in AArch32 and reads no device tree. It uses the legacy mailbox channels, polls without timeouts, and drives USB from the fast interrupt.

So the fork is a catalogue of places where QEMU's model and a real Pi disagree in ways Linux never exercises. Every one was found the same way: trace the guest's device accesses, find the register it spins on, and work out what question it is asking. Most of the fixes are small. The fork counts nine of them as ordinary QEMU bugs that would affect other guests too.

The design principle that decided *how* to fix each one — answer the question rather than model the silicon — has [its own walkthrough](#). This document is the account of the changes themselves.

The road to a desktop, on day one



1. The premise

A sandbox for a compiler

The project began as a testing problem, not an emulation hobby. The RISC OS compiler toolchain, `rosc`, targets **ARMv8-A in 32-bit mode**, tuned for the Cortex-A72 in a Raspberry Pi 4. Its output uses instructions that older ARM cores refuse:

- `CRC32` and `CRC32C`
- the load-acquire and store-release family, `LDA`, `STL`, `LDAEX`, `STLEX`
- the ARMv8 floating-point additions such as `VMAXNM`, `VSEL` and `VCVTA`

The existing test sandbox was RPCEmu, an emulated Acorn Risc PC — at heart a StrongARM, an ARMv4-era core. Every one of those instructions fails there. A plan already existed to teach RPCEmu's interpreter to decode A72 instructions. But that would still be, in the design record's words, a Risc PC with a StrongARM pretending.

The alternative was to emulate the machine the compiler actually targets: **a Raspberry Pi 4, running RISC OS 5, on a real Cortex-A72 in AArch32.**

One ROM for every Pi

RISC OS Open ships one Raspberry Pi ROM image for every model from the Pi 1 to the Pi 4. It detects the SoC at run time and takes different paths in its hardware abstraction layer (HAL). The first trace showed the detection working. The same binary picked the peripheral base `0x3F000000` on QEMU's Pi 2 and `0xFE000000` on its Pi 4.

So the ROM would run on QEMU's older `raspi2b` machine too, and did — as far as the same early hang. That machine was ruled out for a different reason. Its CPU is a Cortex-A7, an **ARMv7** core, which rejects the ARMv8 instructions `rosc` emits. Moving the sandbox from ARMv4 to ARMv7 would not fix the problem it existed to fix.

The routes weighed

Route	Why it was, or was not, taken
RPCEmu, extended	still a Risc PC; the A72 instructions are bolted onto an ARMv4 machine
QEMU raspi2b	runs the ROM, but a Cortex-A7 refuses what the compiler emits
A new emulator	a JIT is available off the shelf, but it is a user-mode core — no MMU, no banked registers, no exception vectors; <i>"months"</i>
QEMU raspi4b	a real A72 translator and a Pi 4 memory map, already built

The new-emulator route was costed honestly and kept as a fallback. It had conditions attached. It would be revisited only if the display path proved structurally unfixable in QEMU, or if QEMU measured slower on real RISC OS code than the RPCEmu interpreter already was. Neither happened.

Speed was not the problem

The speed comparison that made QEMU credible was measured on the i7-12700:

Emulator	Workload	Millions of instructions a second
RPCEmu interpreter	RISC OS desktop, idle	205
RPCEmu dynamic recompiler	RISC OS desktop, idle	~920
QEMU TCG, raspi2b (A7)	two-instruction register loop	2,016
QEMU TCG, raspi4b, A72 in AArch32	two-instruction register loop	2,071
QEMU TCG, raspi2b (A7)	six-instruction load/store loop	465
QEMU TCG, raspi4b, A72 in AArch32	six-instruction load/store loop	579

The QEMU figures come from a 100-byte bare-metal ARM blob, timed by the emulated system timer and printed over the emulated serial port. For scale, a real Pi 4 core is put at roughly 1,000–1,800 — an estimate reasoned from instructions per clock cycle, not a measurement.

The design record reads its own numbers carefully. The register loop is an upper bound, because TCG chains it into a tight host loop with no memory traffic. The load/store figure is the realistic one, though still a synthetic loop rather than OS code. The conclusion holds even so: QEMU is in the class of RPCEmu's dynamic recompiler, several times its interpreter, and **on the right instruction set**.

The CPU half was free

This is the finding that reframed the project. It was widely believed that `raspi4b` is 64-bit only, because the machine hard-codes its CPU. The SoC model names the Cortex-A72 directly, so `-cpu` looks as though it can do nothing there.

It can. When `-cpu cortex-a72,aarch64=off` is given, QEMU records the feature list as **global properties on that CPU type**, applied when any instance of it is created. The SoC never consults `-cpu`. But the A72 it creates still picks up `aarch64=off`. Under TCG that is allowed whenever AArch32 is available at the CPU's highest exception level, which on an A72 it is.

What `aarch64=off` changes

The evidence, from the first afternoon:

Command	First instructions executed
<code>-M raspi4b</code>	AArch64 at 0x80000: the ROM's 32-bit vectors decoded as A64 garbage. Dead
<code>-M raspi4b -cpu cortex-a72,aarch64=off</code>	AArch32 from 0x0, into the ROM. Reaches the HAL
<code>-M raspi3b</code>	AArch64 at 0x80000. Dead
<code>-M raspi2b</code>	AArch32, ROM at 0x10000. Reaches the HAL

The bare-metal benchmark confirmed the rest. In the 32-bit Pi 4 case the BCM2711 peripheral base at `0xFE000000` is live. The serial port at `0xFE201000` and the system timer at `0xFE003004` both answered.

No change to QEMU's CPU emulation was needed to run the ROM. The fork's one commit under `target/arm` came days later, during the HostFS work. That commit itself records that it fixed nothing (chapter 7).

What else QEMU already gave

A test sandbox needs more than a CPU, and QEMU already had most of it. These are the needs the RPCEmu sandbox had identified, mapped onto what QEMU ships:

Need	On QEMU
registers, memory, breakpoints, watchpoints, exact single-step	the gdbstub, plus <code>-accel tcg,one-insn-per-tb=on</code>
halt, continue, machine state	QMP: <code>stop</code> , <code>cont</code> , <code>query-status</code>
screenshots	QMP <code>screendump</code>
snapshots	<code>savevm</code> and <code>loadvm</code> against a qcow2 image
determinism	<code>-icount</code> , with <code>record/replay</code> on top

The design record's one-line answer to "an interpreter and an optional JIT" was that **QEMU is the JIT**. Setting one instruction per translation block, with the gdbstub, gives the interpreter-shaped debugging mode.

The machine half was the work

With the right CPU running the right instruction set from the right address, the ROM starts — and a few thousand instructions later it stops, waiting for something.

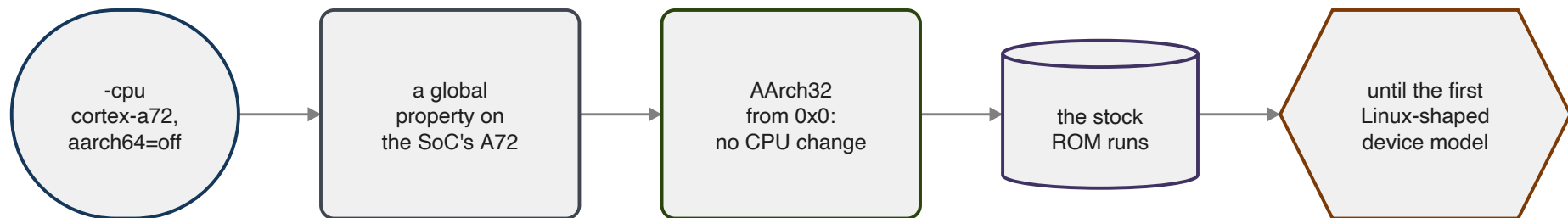
Linux and RISC OS, compared

Every blocker from there on was the same kind of thing: **an assumption in QEMU's Raspberry Pi models that Linux satisfies and RISC OS does not**. Each difference in this table turns into a section of a later chapter.

Linux on a Pi	RISC OS 5 on a Pi
boots in AArch64	boots in AArch32
reads a device tree	reads no device tree; the HAL has a fixed device table
powers devices with property tag 0x28001	powers USB through the legacy mailbox channel 0
secondary cores use a 64-bit spin table	writes a 32-bit entry address to each core's mailbox
waits with timeouts	several waits have no timeout at all
never uses the legacy interrupt controller's FIQ banks	runs USB entirely from the FIQ

A QEMU model can be perfectly good for Linux and still wrong for RISC OS in every row of that table, because Linux never asks those questions.

Where the work actually was



Prior art

The fork was not the first attempt, and the design record lists what came before so that nothing was rediscovered blind:

- **Two public projects from August 2026** boot the same stock 5.30 ROM on QEMU's older Pi 2 machine, with a large patch to QEMU 10.2. They reportedly add a channel-0 power device and a VCHIQ peer, among other fixes. Their channel-0 diagnosis matches the one in chapter 2, which was reached independently. The design record treats that as the strongest evidence either way.
- **A 2013 effort** booted the RISC OS Pi desktop on an out-of-tree BCM2835 branch of QEMU, with both QEMU and the ROM patched.
- **The RISC OS Open forum record** shows hangs at module initialisation (2016), getting no further for want of a working timer (around 2021), and a lock-up when enabling USB power (2025). All of these are consistent with what chapters 2 and 3 found.
- **A maintainer's position**, from 2023, argues against a QEMU fork. It favours a RISC OS HAL for QEMU's generic `virt` machine with VirtIO devices. Nobody has written that HAL. The design record notes it as the view of the people who maintain the OS.

2. First light

Chapter 1 left the stock ROM running on an A72 in AArch32. This chapter follows it through its first second of life. It covers the boot loader QEMU hands it to, the mailbox call it never returns from, and the three secondary cores that were quietly executing garbage.

A loader that thinks it is booting Linux

When QEMU is given a raw image with `-kernel`, its generic ARM boot code assumes a **Linux kernel**. There is no other kind of raw image it knows how to start. For a 32-bit image that means:

- the image is loaded at the start of RAM plus `0x10000`
- a small primary loader is written at address 0; it sets `r0 = 0`, `r1` to a board number and `r2` to a boot-argument pointer, then jumps to the image — the Linux ARM boot protocol
- with no device tree, `r2` points at an ATAGS list at `0x100`
- any secondary cores are reset to a spin stub, which waits to be told where to go

RISC OS ignores all three registers. Its HAL simply begins at the load address, and the ROM image has its HAL at file offset 0. The Linux loader therefore works for RISC OS, but by an accident of layout rather than by design.

One consequence has not been checked. QEMU boots a "Linux" kernel non-secure and enters it at EL2 when the CPU has EL2. It enables secure boot only for the older BCM2835 and BCM2836 SoCs. **INFERRED:** so the RISC OS HAL is probably entered in Hyp mode on this machine. The fork's records never discuss it. The register dumps from the hang show supervisor mode, so the ROM evidently copes.

The fork changes nothing in the loader. It adds one comment to `raspi4b.c`, noting that no device tree ever reaches a `-kernel` guest, so the device-tree edits in that file do nothing for RISC OS.

The first hang: PC = 0x16764

The first run used the released QEMU 11.1.0 binary, before any build from source. It was driven over QMP, with `info registers` sampled across forty seconds. Both Pi machines stopped in the same place:

Machine	R3	R15 (the PC)	Mode
raspi2b	3F00B800	00016764	supervisor
raspi4b + aarch64=off	FE00B800	00016764	supervisor

R3 is the VideoCore mailbox, at the peripheral base plus 0xB800, and it differs between the two runs. **The same ROM binary picked the BCM2836 base on one machine and the BCM2711 base on the other.** So SoC detection worked, and the Pi 4 run really was on the Pi 4 code paths.

The mailbox trace showed the property channel working well. The ROM sent a full tag list — MAC address, serial number, ARM and VideoCore memory, board model and revision, DMA channels, clock rates — and QEMU answered all of it. Then the ROM posted one message on a different channel, and the trace became a single line repeated millions of times:

The mailbox trace

Access	Register	Value
write	MAIL1_WRITE (+0xa0)	0x80
read	MAIL0_STATUS (+0x98)	0x40000000 — <i>EMPTY</i> , 3.9 million times in 12 s

What the ROM was asking

The disassembly around the PC is short. In outline, the HAL's USB start-up does this:

```
message = (1 << 3) << 4 | 0      ; device bit 3, the USB host controller, on channel 0
```

```

write message to MAIL1_WRITE
loop:
    if MAIL0_STATUS has EMPTY set: goto loop    ; no timeout, no retry limit
read the reply

```

A mailbox message carries its channel in the low four bits. **Channel 0 is power management.** The upper 28 bits are a mask of devices the ARM wants powered, and the firmware replies on the same channel with the resulting power state. The HAL is asking the VideoCore to power up the USB host controller, and it waits for an answer with no timeout at all.

QEMU's mailbox model has defined `MBOX_CHAN_POWER` since the mailbox was first modelled. **But nothing was ever attached to it.** Only channel 1 (framebuffer) and channel 8 (property tags) had peers. A message on channel 0 was written into an unmapped slot of the mailbox's address space and dropped. Nothing ever marked a reply as available, so `MAIL0_STATUS` said *empty* forever.

Linux never notices, because it powers devices with the property tag `0x28001` on channel 8 instead. RISC OS uses the legacy interface.

The fix: a peer that says yes

The whole fix is a new 146-line device, `hw/misc/bcm2835_mbox_power.c`. Its logic fits in two functions.

A write is the ARM posting a request. The device keeps the requested mask as the new power state, marks a reply pending and raises the mailbox interrupt:

```

case MBOX_AS_DATA:
    /* bcm2835_mbox checks our pending status before pushing */
    assert(!s->pending);
    /*
     * Everything we model is always on, so the new state is whatever was
     * asked for. Keep it so a read back reports the same mask.
     */
    s->state = value & ~0xfu;
    s->pending = true;
    qemu_set_irq(s->mbox_irq, 1);
    break;

```

A read is the ARM collecting the reply. The reply is the stored mask with the channel number in its low nibble. Reading it clears the pending flag and drops the interrupt:

```
case MBOX_AS_DATA:
    res = s->state | MBOX_CHAN_POWER;
    s->pending = false;
    qemu_set_irq(s->mbox_irq, 0);
    break;
```

Every device QEMU models is always powered, so the reply is simply the request. That is also exactly what the real firmware returns on success.

Two details follow the conventions of QEMU's existing mailbox peers. The `assert` relies on the mailbox device checking for a pending reply before it pushes another message. And the memory region disables QEMU's re-entrancy guard, because the mailbox device calls into this region and then reads back from it. The property channel's peer does the same.

Proving it without RISC OS

A fix that can only be tested by booting a licensed ROM is a poor regression test. So the fork added `tools/mboxtest.s.in`, a bare-metal ARM program of a few dozen instructions. It posts the HAL's exact message — channel 0, device bit 3 — and waits for the reply with a *bounded* spin. It prints the result over the serial port:

The regression test

Build	Output
stock QEMU 11.1.0, on raspi2b or raspi4b	TIMEOUT
with bcm2835-mbox-power	00000080 — channel 0 in the low nibble, the USB bit set

The same test was re-run unchanged when the fork reached the M4 Mac and again on the Intel Mac. It is the one RISC OS-free check the fork carries. Chapter 7 returns to the lack of in-tree tests.

The second hang: three cores running garbage

With channel 0 answered, the boot went further. The volume of device traffic, which the fork used as its progress meter, did not change at all:

Build	MMIO events in ~12 s	Where it stops
stock	1,049,989	mailbox channel 0, PC = 0x16764
+ channel 0	1,049,989	a new spin: read 0xff8000c4

A histogram of the trace by address made it obvious. **Of the last 300,000 MMIO events, 298,606 were reads of 0xff8000c4** — a register in the BCM2711's ARM-local peripheral block.

The RISC OS kernel starts the other three cores the standard Pi way:

1. It writes an entry address into **mailbox 3** of cores 1, 2 and 3, at 0xff80009c , 0xff8000ac and 0xff8000bc .
2. Each secondary core is expected to be sitting in a tiny stub, polling its own mailbox 3. It sees the address, clears the mailbox, and jumps.
3. The new code running on each secondary signals back, and core 0 waits by polling **its own mailbox 1**, at 0xff8000c4 .

How upstream chose the stub

The secondaries never signalled, because they were not running a 32-bit stub. QEMU's `setup_boot()` chose the secondary stub **by SoC generation**, not by what the CPU was doing:

SoC	Stub upstream chose
BCM2836 (Pi 2)	write_smpboot: a 32-bit stub with a board-setup trampoline
anything newer (Pi 3, Pi 4)	write_smpboot64: an AArch64 spin table at 0xd8-0xf0

On a Pi 4 with `aarch64=off` , the three secondaries were reset into an AArch64 stub while running in AArch32. They fetched 64-bit instructions as 32-bit ARM instructions, executed them as garbage, and never parked on their mailboxes. This bug is not specific to RISC OS: **any** 32-bit kernel booted on

QEMU's `raspi3b` or `raspi4b` would meet it. Nobody had hit it because nobody boots 32-bit guests on those machines.

The fix: `write_smpboot32`

Commit `86a43faa32` adds a 32-bit stub for the newer SoCs. It uses the same protocol as the Pi 2 stub, with two differences: it has no board-setup trampoline, which exists only on the BCM2835 and BCM2836, and it takes the local peripheral base as a parameter.

```
static const ARMInsnFixup smpboot[] = {
    { 0xee100fb0 }, /* mrc      p15, 0, r0, c0, c0, 5 ;get core ID */
    { 0xe7e10050 }, /* ubfx    r0, r0, #0, #2      ;extract LSB */
    { 0xe59f5014 }, /* ldr     r5, [pc, #20]       ;load mbox base */
    { 0xe320f001 }, /* 1: yield */
    { 0xe7953200 }, /* ldr     r3, [r5, r0, lsl #4] ;read our mbox */
    { 0xe3530000 }, /* cmp     r3, #0              ;spin while zero */
    { 0xaafffffb }, /* beq     1b */
    { 0xe7853200 }, /* str     r3, [r5, r0, lsl #4] ;clear mbox */
    { 0xe12fff13 }, /* bx      r3                  ;jump to target */
    { 0, FIXUP_BOOTREG }, /* (constant: mailbox 3 read/clear base) */
    { 0, FIXUP_TERMINATOR }
};
```

Reading it line by line:

- **mrc and ubfx** read the core's affinity number from MPIDR and keep the low two bits: 0 to 3.
- **ldr r5** loads a base address from the literal word at the end, which QEMU patches in when it writes the stub.
- **The loop** yields, then reads the word at `base + core × 16` — this core's mailbox 3 read/clear register — and spins while it is zero.
- **str and bx**: writing the value back clears the mailbox, and the core jumps to the address the kernel sent.

The base is the only thing that differs between SoCs. On the BCM2837 the ARM-local block is at `0x40000000`, so the base is `0x400000cc`. The BCM2711 moved the block to `0xff800000`, so its base is `0xff8000cc`.

The selection in `setup_boot()` now looks at the CPU's state rather than only its generation:

```

if (processor_id == PROCESSOR_ID_BCM2836) {
    s->binfo.write_secondary_boot = write_smpboot;
} else if (arm_feature(&cpu->env, ARM_FEATURE_AARCH64)) {
    s->binfo.write_secondary_boot = write_smpboot64;
} else {
    s->binfo.write_secondary_boot = processor_id == PROCESSOR_ID_BCM2838
        ? write_smpboot32_2838
        : write_smpboot32_2837;
}

```

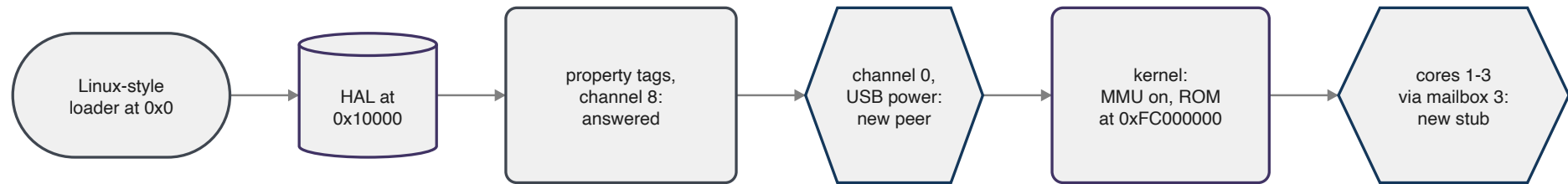
That is the same test QEMU's generic boot code already uses to choose its *primary* loader. The raspi code had simply never applied it to the secondaries.

The result

Build	MMIO events in ~12 s	Where it stops
+ channel 0	1,049,989	secondary-core wait
+ write_smpboot32	6,024	the guest goes quiet

A spinning guest produces a million events. A guest that has stopped producing events has usually taken an exception it cannot recover from. That is where chapter 3 begins.

The first second of boot



3. Tracing beats theorising

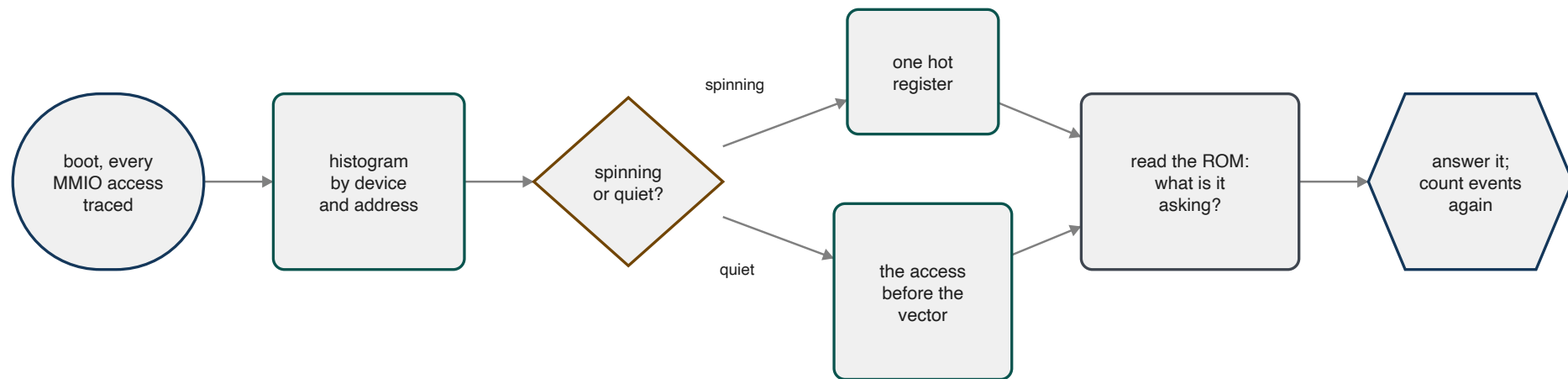
Between the secondary-core fix and the supervisor prompt lay five more blockers. They were found in the space of one afternoon: an I2C controller with three bugs, a system timer wired to nothing, a VCHIQ handshake with a single possible exit, and an unmapped PCIe root complex. This chapter is about those fixes, and about the method that found them. Every attempt to reason from first principles instead turned out wrong.

The method

The design record's own summary of the approach is blunt: *tracing beat theorising every time*. In practice:

1. **Trace every device access.** QEMU's `-trace enable=memory_region_ops_*` logs each MMIO read and write with its device, offset and value.
2. **Histogram by device and by address.** A guest spinning on an unanswered question produces a million events, and nearly all of them hit one register. The blocker announces itself as a single hot address.
3. **If the guest goes quiet instead,** interleave `-d in_asm` (the instructions QEMU translates) with the MMIO trace, and read the seam. The last instruction before the exception vector is the one that faulted.
4. **Read the ROM at that point,** and work out what the guest was asking.
5. **Fix, and measure the MMIO volume again.** A change in volume shows whether the boot moved on.

The loop that found every blocker



Three theories that were wrong

The method earned its place by beating three plausible theories, and each one is worth recording.

Theory	Why it looked right	What the trace showed
display bring-up blocks on GPIO tags	with the channel-0 call patched out of a scratch ROM, the last firmware calls before the hang were display-related tags that QEMU answered with nothing	those tags stop nothing
VCHIQ blocks the display	published work on the Pi 2 path said so	at that stage the guest had never posted to channel 3 at all
the monitor's EDID comes over I2C	an address 0x50 probe on the first I2C bus, where a monitor's EEPROM would sit	0x50 is the HAL probing for a PCF8583 real-time clock

The third cost the most, because it was built. An EEPROM model on the first I2C bus made the probe succeed. RISC OS then read EDID bytes as its CMOS settings. A real Pi has no clock chip there, so **the correct answer is no answer**. The model was removed before it was ever committed.

That theory was wrong for a second reason too. QEMU's own `bcm2835-i2c-test` attaches a temperature sensor at 0x50 on every bus, so any device placed there would have collided with the upstream test. EDID does not travel over I2C on this machine at all. It arrives through a firmware property tag, which is chapter 4's story.

The I2C controller, three bugs deep

After the secondary-core fix the guest went quiet at 6,024 events. The instruction trace showed a byte-sized store to the I2C controller's FIFO register at offset 0x10, followed immediately by the data abort vector.

Bug 1: a byte port that refused bytes

The BSC I2C controller's FIFO is a byte port. The HAL writes it with `STRB`, which is the natural instruction for a byte port. QEMU's model declared every register 4-byte-only, and QEMU's memory core turns a disallowed access size into a bus error, which the guest takes as a data abort.

Commit `d742c3b617` widens the access sizes:

```
.valid = {
    .min_access_size = 1,
    .max_access_size = 4,
},
/*
 * The FIFO is a byte port, and guests do use STRB/LDRB on it. Accesses
 * must reach the handlers at the size the guest used: letting the core
 * synthesise a byte access out of a 32-bit one would turn a byte write
 * into a read-modify-write, and reading the FIFO pops it.
 */
.impl = {
    .min_access_size = 1,
    .max_access_size = 4,
},
```

The subtle half is `impl`. QEMU separates the sizes a guest may use (`valid`) from the sizes the handler implements (`impl`). If only `valid` had been widened, QEMU would have emulated each byte write as *read the word, change one byte, write the word*. For a FIFO both halves are wrong: the read pops a byte, and the write sends three bytes the guest never sent.

Bug 2: sub-word addresses

Allowing byte access was only half the job. A byte access at offset 1, 2 or 3 of a register fell through to the "bad offset" case. On hardware it simply addresses one byte of the 32-bit register. Commit `57724dbd8e` splits every address into a word and a byte within it, and merges the written byte into the rest:

```
hwaddr reg = addr & ~3;
unsigned shift = (addr & 3) * 8;

if (size < 4) {
    uint32_t mask = ((1u << (size * 8)) - 1) << shift;
```

```

switch (reg) {
case BCM2835_I2C_FIFO:
    if (shift) {                /* only the low byte is the data port */
        return;
    }
    break;
case BCM2835_I2C_S:            /* write-1-to-clear: unwritten bytes clear nothing */
    writeval = (writeval << shift) & mask;
    break;
default:                        /* a plain register: merge into the bytes not written */
    writeval = (bcm2835_i2c_shadow(s, reg) & ~mask)
        | ((writeval << shift) & mask);
    break;
}
}

```

Two registers need special care. The FIFO's upper bytes are not part of the port, so reading them returns zero without popping, and writing them does nothing. The status register is write-1-to-clear. For a plain register, the bytes the guest did not write are filled in from the current value. Doing that here would clear whatever bits happened to be set.

Bug 3: a state machine that dropped the message

With byte access working, the guest spun again, on the status register. It read `S = 0x57`: *transfer active* and *done* both set. The HAL's transfer loop waits for *transfer active* to clear. QEMU cleared it only when the data length counted down to zero, and the count never moved.

The documented way to drive the controller is:

1. set the slave address `A`
2. set the data length `DLEN`
3. fill the FIFO
4. **then** set `ST` in the control register to start

The HAL does exactly that. QEMU's model had two bugs that both break this order:

- **The start condition fired on I2CEN as well as ST.** Every write that merely *enabled* the controller began a transfer, and with `DLEN` still zero it ended that transfer at once. By the time the guest set `ST`, the model had opened and closed two transfers to the wrong address.
- **There was no transmit FIFO.** Bytes written to the FIFO went straight to `i2c_send()`, and were dropped if no transfer happened to be active. The HAL's bytes, written before `ST`, vanished.

Commit `59e0d5b5ea` requires `ST` to start a transfer, and adds the hardware's 16-byte transmit FIFO with a drain:

```
if ((writeval & BCM2835_I2C_C_ST) && (writeval & BCM2835_I2C_C_I2CEN)) {
    bcm2835_i2c_begin_transfer(s);      /* which drains anything already queued */
    ...
}

static void bcm2835_i2c_tx_drain(BCM2835I2CState *s)
{
    while ((s->s & BCM2835_I2C_S_TA) && s->dlen > 0 &&
           !fifo8_is_empty(&s->tx_fifo)) {
        if (i2c_send(s->bus, fifo8_pop(&s->tx_fifo))) {
            s->s |= BCM2835_I2C_S_ERR;
        }
        s->dlen -= 1;
    }
    bcm2835_i2c_update_tx_status(s);
    if ((s->s & BCM2835_I2C_S_TA) && s->dlen == 0) {
        bcm2835_i2c_finish_transfer(s);
    }
}
```

The drain runs from two places. Starting a transfer drains bytes queued beforehand, and a FIFO write drains bytes queued afterwards, so both orders work. Clearing the FIFO and finishing a transfer both empty it. QEMU's own I2C test still passes. The FIFO is new migrated state, so the device's migration version goes from 1 to 2.

With the state machine fixed, the probe at `0x50` fails cleanly: *done* and *error* are set, and *transfer active* is clear. The guest moves on to `0x68` on the *second* I2C bus, where a Pi 4 would carry a DS1307-style clock, and gets the same honest silence.

The MMIO volume, fix by fix

Build	MMIO events in ~12 s	Where it stops
stock	1,049,989	mailbox channel 0
+ channel 0	1,049,989	secondary-core wait
+ 32-bit secondary stub	6,024	I2C FIFO byte store: data abort
+ I2C byte access	5,685,747	the 0x50 probe answered by nobody
+ EDID EEPROM (abandoned)	5,643,283	status spin, S = 0x57

Two firmware tags, answered anyway

Two property-tag fixes from the same hour were the "display bring-up" theory's leftovers. They turned out not to be blockers, but the answers were wrong and are now right:

- **f87dd5f72**: `SET/GET_TOUCHBUF` and `SET/GET_GPIOVIRTBUF` had fallen through to a zero-length reply. Each carries an address the guest gives the firmware and may ask for back — the touchscreen's state page, and the virtual GPIO page for the activity LED. The model now stores and returns it.
- **42de159040**: `SET/GET_GPIO_STATE` is now answered at its documented length, with the expander pins reported off. A guest that checks the reply length would otherwise see a failure.

The clock that never ticked

With I2C out of the way, the guest ran on but never advanced. Every timed wait was infinite. The trace said it plainly, over the whole run: **one** write to a system-timer compare register, **one** timer expiry, and **no** interrupt acknowledgement, ever.

The cause was in the SoC model. A BCM2711 has two interrupt controllers: the old BCM2835-style controller and a GIC-400. A Pi 4 guest uses the GIC. QEMU's `bcm2838_realize()` re-routes each peripheral's interrupt line from the legacy controller to the GIC: UART0, AUX, I2C, mailbox, SDHOST,

EMMC, MPHI, DWC2 and DMA. **The system timer was missing from that list.** Its four compare outputs were wired once, in the shared BCM2835 code, to the legacy controller, and nothing ever forwarded them.

RISC OS uses compare 1 as its centisecond tick, and enables GIC interrupt 97 for it. That is SPI 65: the legacy VideoCore interrupts appear at SPI 64 plus their number. The timer expired once, set its status bit, and raised a line into a controller nobody was listening to. The OS then waited for a clock that could not advance.

Commit `57651d54ce` adds the missing loop:

```
/*
 * Connect the system timer to the interrupt controller. Without this its
 * compare outputs reach only the legacy interrupt controller, which a
 * guest driving a BCM2711 through the GIC never looks at -- the timer
 * fires, sets its status bit, and nobody is told.
 */
for (int n = 0; n < 4; n++) {
    sysbus_connect_irq(SYS_BUS_DEVICE(&ps_base->systmr), n,
                      qdev_get_gpio_in(gicdev, GIC_SPI_INTERRUPT_SYSTIMER0 + n));
}
```

Before and after

	Before	After
timer expires in ~20 s	1	1,335
timer interrupt acknowledgements	0	1,335
GIC acknowledge reads for interrupt 97	0	2,953 in 45 s

This is the bug the RISC OS forum's history had described years earlier, as a boot that got no further for want of a working timer. It affects any guest that uses the system timer through the GIC on `raspi4b`. With it fixed, RISC OS services its tick, dispatches SWIs, runs module code in RAM, starts probing the USB controller — and posts its first message to mailbox **channel 3**.

VCHIQ: a wait with exactly one exit

Channel 3 is VCHIQ, the message queue between the ARM and the VideoCore GPU. QEMU has never had a device on it. The guest posts the bus address of a `slot zero` structure — the queue's shared memory — and waits.

The wait cannot be failed from outside. The connect call blocks on a counting semaphore with no timeout, no deadline and no register the guest reads. No value QEMU could return and no error it could inject would end it. The only exit is a real `CONNECT` message written into shared memory, followed by a doorbell interrupt.

The module making that call was not the video driver either. The ROM initialises its modules in this order, and it is **BCMSound** whose initialisation calls `VCHIQ_Connect` unconditionally:

```
RTSupport → USBDriver → DWCDriver → XHCIDriver → VCHIQ → BCMSound → ScreenModes → BCMVideo
```

The display code never ran at all. The boot was stuck in the sound driver.

Reading the structure out of a live guest

Before a line of the peer was written, the slot-zero structure was read out of the stalled guest's memory:

Field	VideoCore side, at +0x20	ARM side, at +0x194
initialised	0	1
slot_first / slot_last	2 / 32	34 / 64
tx_pos	—	8
slot_queue[0]	—	34

Slot 34 already held a `CONNECT`, written by the guest and never read. The peer derives the stride between the two sides from the structure's own `slot_zero_size` field rather than hard-coding it. A debug array sits at the end of the structure, and its length is a build option.

What the peer does

Commit `1013487c14` adds `hw/misc/bcm2835_vchiq.c`. It is not a VideoCore; it completes the handshake and nothing more:

1. set the VideoCore side's `initialised` flag
2. fill the slot queue, which the guest zeroed and expects the VideoCore to own
3. arm its own trigger, and queue a `CONNECT` of its own
4. ring the VideoCore-to-ARM doorbell, at mailbox base `+0x40`, on SPI 34 (GIC interrupt 66, one above the mailbox)
5. then **refuse every service the guest opens**: `AUDS`, `GCMD`, `DISP`, `TVSV`

Refusing is the correct answer, not a shortcut. Accepting `AUDS` walks BCMsSound into a blocking message queue, and accepting `TVSV` arms two more spins with no timeout. A clean refusal leaves the driver's `GPUModeAvailable` at zero. Mode setting then falls back to the property channel, which QEMU already models completely.

The tags that followed

The proof is the sequence of property tags that follows, which had never appeared before:

Tag	Meaning
0x00010006	get VideoCore memory: BCMVideo's init, running at last
0x00030020	get the EDID block
0x00048001	release the framebuffer
0x00048003–0x00048007, 0x00040008	size, depth, pixel order, alpha, pitch
0x00040001	allocate the framebuffer
0x00040002, 0x0004800b	unblank; set the palette

A doorbell that must be read to clear

One doorbell detail matters more than its size suggests:

```
case VCHIQ_BELL0:
    /*
     * Read to clear. The guest's VCHIQ device declares no device-specific
     * interrupt clear, so this read is the only thing that lowers the
     * line -- and the interrupt is level triggered, so failing to lower
     * it here would put the guest in an interrupt storm.
     */
    val = s->bell0;
    s->bell0 = 0;
    qemu_set_irq(s->bell_irq, 0);
    return val;
```

The doorbells live inside the address range the mailbox device already claims. So they are overlaid at a higher priority, rather than by reaching into the mailbox model.

Review fixes, the same evening

Commit `baddb065d3` came from review:

- **An over-strict address check went.** The peer had required the slot-zero address to use the `0xC0000000` bus alias. That holds on a Pi 2 and later, but a Pi 1 uses `0x40000000`. The device is built into every raspi machine through the shared peripherals code, so the check would have broken Pi 1 guests.
- **Every number read from the guest is now bounded:** the slot-zero size (an unsigned subtraction that could underflow), the slot range, the queue entries and each message's size field. None of these could reach host memory, because every access goes through the guest's DMA address space. But a confused guest could have had the device scribble over its own RAM.

The peer later grew well past a refuser. It accepts `AUDS` for sound and `DISP` for the mouse pointer, and it is 1,581 lines at the head used here — see the [graphics and sound walkthrough](#). Its header comment still says it refuses every service.

PCle: "nothing here", not "the bus is on fire"

With VCHIQ answered, RISC OS completed its module initialisation, allocated a framebuffer, and reached its supervisor prompt at 15:59. It also printed a data abort on its own console.

The faulting registers

The faulting addresses were in the BCM2711's PCIe root complex:

Offset	Register	Part of
0x9210	PCIE_RGR1_SW_INIT_1	reset
0x4008	PCIE_MISC_MISC_CTRL	set-up
0x402c	PCIE_MISC_RC_BAR1_CONFIG_LO	the BAR probe

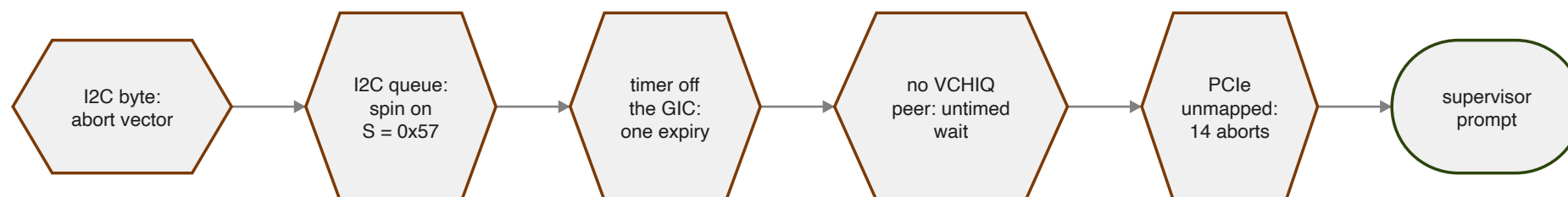
That is the documented reset-and-probe sequence. RISC OS's `PCI_Init` runs it only on a Pi 4, looking for the VL805 USB 3 controller behind the root complex. Nothing is mapped there in QEMU, so each access took an *external* abort — a bus error rather than a translation fault.

Commit `1a22c9b0b1` maps an unimplemented-device stub over the region, at low priority:

```
object_initialize_child(OBJECT(s), "bcm2838-pcie", &s->pcie,
                       TYPE_UNIMPLEMENTED_DEVICE);
qdev_prop_set_string(DEVICE(&s->pcie), "name", "bcm2838-pcie");
qdev_prop_set_uint64(DEVICE(&s->pcie), "size", BCM2838_PCIE_SIZE);
sysbus_realize(SYS_BUS_DEVICE(&s->pcie), &error_fatal);
memory_region_add_subregion_overlap(&s->peri_low_mr, BCM2838_PCIE_OFFSET,
                                   sysbus_mmio_get_region(SYS_BUS_DEVICE(&s->pcie), 0), -1000);
```

An unimplemented device reads as zero, and the guest reads zero as *link down*. It gives up quietly. The stub does not pretend to be a root complex. As the commit puts it, it is the difference between *there is nothing here* and *the bus is on fire*. Aborts over a 30-second boot fell from **14 to 1**. The remaining one was the network driver, and chapter 4 deals with it.

Five blockers, five symptoms



About 2,900 interrupts were now serviced in a 30-second boot, from the tick and the doorbell. They were real interrupts, where the OS genuinely depends on them, and functional answers everywhere else.

4. QEMU is the firmware

A real Raspberry Pi does not start the ARM first. The VideoCore GPU boots, runs Broadcom's closed firmware, reads `config.txt` from the card, loads the OS image and a CMOS settings file into memory, and only then releases the ARM cores. From that point the firmware stays running. It answers mailbox calls, reports the monitor's EDID and hands out the framebuffer.

Under QEMU none of that firmware runs. The kernel image is loaded straight into memory with `-kernel`, and every job the firmware would have done falls to QEMU — either as a device model's answer to a question, or as a file loaded at the right address. This chapter covers the firmware duties that stood between the supervisor prompt and a desktop.

Firmware duties, and who does them now

Firmware duty on a real Pi	How the fork provides it
load the OS image	<code>-kernel RISCOS.IMG</code>
load the CMOS settings file	<code>-device loader</code> , with a blob built by <code>mkcmos.py</code>
answer mailbox channel 0 (power)	the new power peer (chapter 2)
answer VCHIQ on channel 3	the new VCHIQ peer (chapter 3)
answer the EDID property tag	an EDID block built from a mode table
framebuffer, memory, board and clock tags	already in QEMU's property model

The CMOS blob

A Pi has no CMOS chip. The HAL reads its non-volatile settings from a blob the firmware leaves in memory **immediately after the OS image**. It reads the blob before the MMU is on and checks a version word. If the word is out of range, it blanks every setting to `0xFF`.

Stock QEMU leaves nothing there, so every boot was a machine with no configuration. It could reach the supervisor prompt, but never the desktop.

Where the blob goes

The address is not a constant. The ROM file begins with the HAL, whose image is `0x10000` bytes. The OS image follows at file offset `0x10000`, with a header that starts with the magic word `OSIm`, then flags, then the OS image size. The HAL works out the CMOS address from that size:

Where the ROM is loaded	Address
QEMU -kernel: image at <code>0x10000</code>	$0x10000 + \text{HAL } 0x10000 + \text{OS image } 0x4F0000 = \mathbf{0x510000}$
a real Pi: image at <code>0x8000</code>	$0x8000 + \text{HAL } 0x10000 + \text{OS image } 0x4F0000 = 0x508000$

The second row matches the real firmware's own `config.txt`, which loads the CMOS file at `0x508000`. That agreement confirms the arithmetic. Under QEMU, one more option supplies the blob:

```
-device loader,file=cmos.bin,addr=0x510000,force-raw=on
```

That was the last thing between the supervisor prompt and the desktop. **It needed no device model at all** — only the right bytes at the right address.

Built from source, not transcribed

`tools/mkcmos.py` (commit `106a0b5554`) builds the blob without a single hand-typed setting:

- **The layout** comes from RISC OS's own `hdr/CMOS` header. The tool simulates the assembler's storage-map directives over that file to compute each setting's byte address.
- **The defaults** come from parsing the kernel's `DefaultCMOSTable` and evaluating its expressions.
- **The load address** comes from the ROM's own `OSIm` header.
- **A cross-check** compares the computed layout with the address comments in the header. 72 of 73 agree, and the tool reports the one that does not.

Later options

Later options removed the need for a RISC OS source checkout:

Option	What it does
<code>--symbols cmos-symbols-530.json</code>	uses a committed, pre-resolved symbol table instead of parsing headers
<code>--base CMOS</code>	starts from the SD card's own 2,048-byte CMOS file, which already sets the monitor type to EDID
<code>--unplug EtherGENET</code>	resolves a module name to its ROM chunk number by walking the ROM's module chain
<code>--filesystem 192</code>	boots from SDFS
<code>--language 1</code>	boots to the supervisor prompt on purpose; the stock default, 11, is the desktop

The checksum trap

One behaviour is worth knowing before anyone builds a blob by hand. **A valid checksum makes the kernel skip its CMOS reset entirely.** So the blob must carry every setting, not just the one being changed. A blob of zeros with a correct checksum boots to a black screen. The problem it was meant to fix is gone, and so is every other setting.

Where the blob sits



A network driver that crashes on no network

With a CMOS blob in place, one data abort was still left from chapter 3. It was never the emulator's fault.

RISC OS 5.30's ROM carries **EtherGENET**, the driver for the Pi 4's on-chip GENET Ethernet controller. QEMU does not model GENET. With no controller to find, the driver's attach routine correctly reports that no device exists — but it leaves its interface pointer NULL. Code booked during the module's start-up later calls through that pointer. The result is a data abort at `&FC3FB800`, on address `0x18`, which takes down the whole boot.

This is a bug in the driver, and no device model can prevent it, because no device is involved. Two fixes were built:

Two ways to keep EtherGENET out

Fix	How	Needs
the CMOS unplug bit	<code>mkcmos.py --unplug EtherGENET: ROM chunk 106</code> , so byte &82 gains bit 0x04; the kernel never starts the module	the committed symbol table
a patched ROM	<code>tools/patch-rom-nogenet.py</code> finds the module by title, makes its init return success, and zeroes its service and SWI entries	nothing; source-free

A third change keeps the driver's first register read from aborting. Commits [418847ac82](#) and [5a61a7881c](#) put an unimplemented-device stub over the GENET registers. The stub is at `0xfd580000`: that is where a BCM2711 has them, and it is the address the HAL hands the driver from its fixed device table. The stub reads as zero, which the driver takes as an unknown controller. On its own it does not save the boot. The unplug bit or the patch is still required.

With the blob and the unplug bit, RISC OS 5.30 reached **the desktop from the ROM alone, at 18:14 on day one**.

The card was on the wrong controller

The next step was booting from the RISC OS Open SD card image rather than the ROM alone. With `--filesystem 192` and the image attached as `-drive if=sd`, RISC OS got as far as a desktop error box saying the drive was empty.

The BCM2711's SD controllers

The card was there. QEMU had plugged it into the wrong controller. A BCM2711 has three SD host controllers:

Controller	On a real Pi 4	What RISC OS does with it
legacy EMMC	the WiFi chip	marks it as the WiFi slot
SDHOST	unused for the card	never registers it
EMMC2 , at 0x340000	the removable card	looks for the card here

QEMU's `bcm2838_peripherals` pointed the machine's `sd-bus` alias at the GPIO block, whose pin mux only ever moves a card between the legacy EMMC and SDHOST. So every card given to `raspi4b` sat one controller over from where the guest looked. Commit `a7ea3f48da` is one line:

```
object_property_add_alias(OBJECT(s), "sd-bus", OBJECT(&s->emmc2), "sd-bus");
```

After it, RISC OS mounts the card as `SDFS::RISCOSPi`, runs `!Boot` from it, and reaches the desktop. The change affects any `raspi4b` guest that uses the card. Linux's own device tree for the Pi 4 also puts the card on EMMC2.

QEMU has requirements of its own for the image: it must be writable and, at 2 GiB or under, exactly a power of two in size. The RISC OS Open image was truncated to 2 GiB.

Reading the card faster

Booting from the card exposed a cost. A multiple-block read arrives at QEMU's SD card model one 512-byte block at a time. Each block was a synchronous read of the backing file, a round trip through the block layer's thread pool, and a RISC OS boot makes about **32,000** of them.

Commit `e48ee2c73a` adds a 64 KiB read-ahead buffer:

```
if (addr < sd->readahead_addr ||
    addr + len > sd->readahead_addr + sd->readahead_len) {
    int64_t avail = blk_getlength(sd->blk) - addr;
```

```
uint32_t n = MIN(SD_READAHEAD_SIZE, MAX(avail, 0));
...
if (n < len || blk_pread(sd->blk, addr, n, sd->readahead, 0) < 0) {
    fprintf(stderr, "sd_blk_read: read error on host side\n");
    return;
}
sd->readahead_addr = addr;
sd->readahead_len = n;
}
memcpy(sd->data, sd->readahead + (addr - sd->readahead_addr), len);
```

Writes, resets and media changes discard the buffer. It is not migrated, because it refills itself. The card-reading phase of the boot went from about **seven seconds to five**. The rest of that phase is a stall of about a millisecond per block, which has been measured but is **still unexplained**.

The monitor that was not there

The first desktop was 640×256, the lowest-resolution mode RISC OS has. The display driver asks the firmware for the monitor's EDID with property tag `0x00030020`, `GET_EDID_BLOCK`. QEMU did not answer it, so the read failed as *no acknowledge*. The screen-mode module then fell back to the kernel's oldest numbered modes.

An 800×600 monitor

Commit `70522977c7` answers block 0 with an EDID 1.3 block for a digital 800×600 display. It carries:

- VESA 800×600 at 60 Hz as the preferred timing
- 640×480 and 800×600 among the established timings
- a range-limits descriptor that admits them
- a checksum, computed as the block is handed over

Any other block number returns a non-zero status. That is how the guest learns there is no extension block. With a CMOS whose monitor type is EDID, the desktop came up at 800×600.

Wide modes, and the real gate

RISC OS builds its whole mode list from the EDID, so the sizes a user can choose were decided by 127 hand-written bytes. Commit [7ef62fa51f](#) replaced them with a table, and `-global bcm2835-property.mode=WxH` picks the preferred entry:

```
static const BCM2835EdidMode bcm2835_edid_modes[] = {
    { 640, 480, 25175, 160, 16, 96, 45, 10, 2 }, /* DMT */
    { 800, 600, 40000, 256, 40, 128, 28, 1, 4 }, /* DMT */
    { 1024, 768, 65000, 320, 24, 136, 38, 3, 6 }, /* DMT */
    { 1280, 720, 74250, 370, 110, 40, 30, 5, 5 }, /* CEA-861 */
    { 1280, 800, 71000, 160, 48, 32, 23, 3, 6 }, /* CVT-RB */
    { 1280, 1024, 108000, 408, 48, 112, 42, 1, 3 }, /* DMT */
    { 1440, 900, 88750, 160, 48, 32, 26, 3, 6 }, /* CVT-RB */
    { 1600, 1200, 162000, 560, 64, 192, 50, 1, 3 }, /* DMT */
    { 1920, 1080, 148500, 280, 88, 44, 45, 4, 5 }, /* CEA-861 */
    { 1920, 1200, 154000, 160, 48, 32, 35, 3, 6 }, /* CVT-RB */
};
```

Each row gives width, height, pixel clock in kHz, then the horizontal and vertical blanking, front porch and sync widths. Nothing behind the table is real, so the timings only need to be well formed and consistent. They are the standard figures anyway, because a plausible block is easier to debug than an invented one.

Advertising 1920×1200 changed nothing at first. **The range-limits descriptor was the real gate.** RISC OS checks every timing against it before offering a mode. The old block capped the pixel clock at 50 MHz and the line rate at 50 kHz, which silently discards anything above about 800×600. 1920×1200 needs 154 MHz and 75 kHz:

```
d = edid + 90;
d[3] = 0xfd;
d[5] = 50; /* 50-75 Hz vertical */
d[6] = 75;
d[7] = 15; /* 15-200 kHz horizontal */
d[8] = 200;
d[9] = 60; /* 600 MHz pixel clock */
```

The same commit fixed a pointer bug that only showed at large sizes. The VCHIQ display service's `GetInfo` query had answered with the *current* framebuffer. During boot that is 640×480, the mode the machine passes through before it settles. The guest asks once and scales its pointer by the answer for the whole session. At 800×600 the error was a barely visible 1.25×. At 1920×1200 it made the pointer 10×10 pixels instead of 32×32, confined to a third of the screen and moving in three-pixel steps. The query now answers with the EDID's preferred timing, which does not change.

Limits: the table tops out at 1920×1200, and there is no CTA-861 extension block. The standard-timing encoding can only express widths that are multiples of eight, up to 2,288. A plan to go to 3840×2160 has not been built.

The one firmware call still not implemented

`SET_CLOCK_RATE` (`0x00038002`) is still not implemented. Commit `aeffd4be90` makes it log which clock is being set. On a boot the only requests are for clock 2, the UART, at 3 MHz, and clock 3, the ARM, at 700 MHz. The machine reaches the desktop without an answer to either.

5. Input and network through a FIQ

By the evening of day one the machine had a desktop and a disc, but no way to touch either. A `usb-kbd` had been attached to every run for hours, and RISC OS never enumerated it. This chapter is the longest single investigation in the fork. It ends with a keyboard, a tablet pointer and a network, and with RISC OS's USB stack running from the fast interrupt, exactly as it does on silicon.

The symptom

The Pi 4's DWC2 USB controller was clearly alive. QEMU's model ran, and it produced start-of-frame interrupts at a steady rate. But **it never programmed a single host channel**, and without host channels no USB transfer ever happens.

The trace looked healthy, and that was misleading. `usb_dwc2_attach`, the bus start and the flood of start-of-frame events are all the *model's* own reset-time work. The design record's lesson from this: **the only trace that proves the guest's driver has run is a write to GINTMSK or GAHBCFG** — the controller's interrupt mask and bus configuration. Those writes never came.

A trace that hid its own evidence

An earlier reading had cleared one suspect. The Pi 4's ARM control block keeps an MPH1 alias at `0xb200`. It looked harmless, because no writes to the `0xfe00b3xx` range appeared anywhere in the trace.

That reading was wrong, for a reason worth remembering:

QEMU logs an access through an alias against the target device's own offsets.

The writes were in the trace. They appeared as `mph1` offsets `0x110`, `0x120`, `0x124` and `0x128`. Through the alias at `0xb200` those are `0xb310` to `0xb328`, the enable and disable registers of the BCM2711's **FIQ controller** at `0xb300`. They were being swallowed.

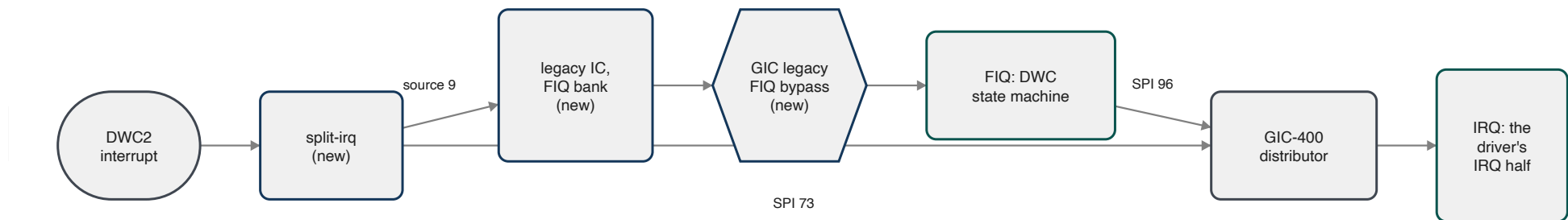
What RISC OS does with USB on a Pi 4

Reading the RISC OS sources explained where those writes were meant to go. In outline:

- **The DWC driver always runs the host controller from its FIQ handler.** It claims the MPHI device's interrupt vector, not the USB one. It arms the controller by enabling FIQ source 9, and nothing ever enables the USB *IRQ* at the GIC.
- **On a Pi 4, the HAL's FIQ enable** writes `1 << 9` to the legacy controller's FIQ enable register at `0xfe00b310`. Its FIQ-source routine reads the pending registers, one with a byte-sized `LDRB`, and scans them for the highest set bit.
- **A Pi 4 has no MPHI block**, so the FIQ handler passes work to the IRQ side through the GIC itself. It sets the pending bit for SPI 96 in `GICD_ISPENDR`, and the IRQ handler clears it through `GICD_ICPENDR`. QEMU's GIC already handles that part.
- **The HAL's GIC set-up** explains how the FIQ is meant to arrive. The boot stub leaves the GIC unable to raise FIQs from non-secure state. So the HAL leaves the GIC's *interrupt signal bypass* enabled, and takes its FIQs from the BCM2711's legacy controller instead.

So on real hardware the path runs: the DWC2's interrupt line → the BCM2711 legacy interrupt controller's FIQ bank for core 0 → the GIC-400's legacy nFIQ input → the CPU, through the bypass. **QEMU had none of that path.**

The FIQ path, as modelled



Piece 1: the BCM2711 legacy interrupt controller

The datasheet (BCM2711 ARM Peripherals, §6.5.3) gives the whole register map. Commit `873dccc24c` adds `hw/intc/bcm2838_ic.c`, about 200 lines, which follows it closely:

Offset from ARMC	Contents
$+0x200 + 0x40 \times n$	the IRQ bank for core n : PENDING0–2, SET_EN_0–2, CLR_EN_0–2
$+0x300 + 0x40 \times n$	the FIQ bank for core n , laid out the same way
$+0x230$	IRQ_STATUS0–2, the raw lines, in the first bank only
$+0x3f0$	SWIRQ_SET/CLEAR, left to the existing MPH1 model

Eight banks of enables cover the same 80 sources. A bank's output is the OR of its enabled sources, and the pending registers show only enabled ones. There is nothing to acknowledge: a source stays pending until the device drops its line.

```
static bool bcm2838_ic_bank_active(BCM2838ICState *s, int bank)
{
    return (s->level[0] & s->enable[bank][0])
        || (s->level[1] & s->enable[bank][1])
        || (s->level[2] & s->enable[bank][2] & WORD2_SOURCES);
}

static void bcm2838_ic_update(BCM2838ICState *s)
{
    for (int bank = 0; bank < BCM2838_IC_NUM_BANKS; bank++) {
        qemu_irq out = bank < BCM2838_IC_NUM_CORES
            ? s->irq[bank] : s->fiq[bank - BCM2838_IC_NUM_CORES];

        qemu_set_irq(out, bcm2838_ic_bank_active(s, bank));
    }
}
```

Word 2 of each bank is special. It holds sixteen more sources, then two summary bits saying whether words 0 and 1 have anything pending, then a copy of the core's own line. Only the sixteen sources gate the output.

The model is mapped at priority 1 over the `0xb200` region. That puts it above both the BCM2835 controller the shared code maps there and the MPH1 alias, which between them had hidden the registers. It accepts byte-sized reads, for the HAL's `LDRB`.

Piece 2: the GIC's legacy FIQ input

A GIC-400 has a legacy nIRQ and nFIQ input for each CPU, beside the distributor. Its CPU interface passes the legacy signal straight to the processor whenever it is not signalling that class of interrupt itself. That is the *interrupt signal bypass*. Commit `54eac9c9fb` adds a `legacy-fiq` input per CPU to QEMU's GIC:

```
static inline bool gic_legacy_fiq_bypassed(GICState *s, int cpu, bool virt)
{
    uint32_t ctrlr = s->cpu_ctrlr[cpu];
    bool own_fiqs = (ctrlr & GICC_CTLR_EN_GRP0) && (ctrlr & GICC_CTLR_FIQ_EN);

    return !virt && s->legacy_fiq[cpu] && !own_fiqs;
}
```

It is folded into both places where the GIC drives a CPU's FIQ line. That includes the early exit taken **while the GIC is disabled altogether**:

```
if (!gic_irq_signaling_enabled(s, cpu, virt,
                              GICD_CTLR_EN_GRP0 | GICD_CTLR_EN_GRP1)) {
    qemu_irq_lower(irq_lines[cpu]);
    qemu_set_irq(fiq_lines[cpu],
                 gic_legacy_fiq_bypassed(s, cpu, virt));
    continue;
}
...
qemu_set_irq(fiq_lines[cpu],
             fiq_level || gic_legacy_fiq_bypassed(s, cpu, virt));
```

Three details matter. QEMU already masks the bypass-disable bits out of `GICC_CTLR`, so the bypass counts as always permitted, which is also what the register reads back as. The bypass is gated off for virtual CPU interfaces. And the input's level travels in a migration subsection that is sent only when an input is high, so existing snapshots of other machines are unaffected.

Piece 3: the wiring

In `bcm2838_peripherals.c` the DWC2 line now goes through a `split-irq`. One leg goes to the GIC as before, and the other to legacy source 9. In `bcm2838.c` the legacy controller's four FIQ outputs go into the GIC's new inputs:

```
/*
 * The legacy controller's FIQs reach each core through the GIC's
 * bypass. Its IRQ outputs are left unconnected: that bypass only opens
 * with the GIC's own signalling off, which no guest of this SoC does.
 */
for (int n = 0; n < BCM283X_NCPUS; n++) {
    qdev_connect_gpio_out_named(DEVICE(&ps->ic), BCM2838_IC_FIQ_OUT, n,
                               qdev_get_gpio_in_named(gicdev, "legacy-fiq", n));
}
```

Chapter 6 adds two more splits of the same shape, for the SMI vsync latch and the ARM timer.

Piece 4: two DWC2 bugs that would have wedged the machine

Two bugs in QEMU's DWC2 model would have hung the guest the moment FIQs started arriving. They were found by reviewing the plan before the FIQ path was switched on, then confirmed in the code. Commit `622a7bde8c` fixes both.

The host-channel interrupt summary ignored its mask. `GINTSTS.HCHINT` is the live OR of the channel interrupts and their mask. The model re-evaluated it only when a channel raised or dropped its own bit, never when the mask changed. RISC OS's FIQ state machine defers every channel it does not handle itself by *masking* that channel, and it relies on the line falling as a result:

```

if (addr == HAINMSK) {
    /*
     * GINTSTS.HCHINT is the live OR of HAIN & HAINMSK, so a new mask
     * moves it as much as a channel event does. A driver that defers a
     * channel interrupt by masking it relies on the line falling here.
     */
    if (s->haint & s->haintmsk) {
        dwc2_raise_global_irq(s, GINTSTS_HCHINT);
    } else {
        dwc2_lower_global_irq(s, GINTSTS_HCHINT);
    }
}

```

The interrupt level was a static. `dwc2_update_irq` remembered the last level it had driven in a function-local `static int`. That one variable was shared by every DWC2 controller in the process and never reset with the device. The level now lives in the device state. It is not migrated; `post_load` recomputes it from the registers.

The evidence that it worked

In order:

1. `FIQ0_SET_EN_0` reads back `0x200`: bit 9, enabled.
2. `info registers` catches the CPU in FIQ mode, with the PC inside the DWC driver.
3. The DWC2 trace shows an enumeration's control transfers, followed by the keyboard's interrupt endpoint being polled.
4. `F12` opens the desktop's command line.

A keyboard and a mouse behind a `usb-hub` enumerate too, and the pointer goes where QMP's `input-send-event` sends it. The FIQ fires once per USB start-of-frame, about **a thousand times a second**. The design record notes that this is the design, not a leak.

One rig lesson: a bare `-device usb-kbd` on this machine is placed behind a hub. Adding `bus=usb-bus.0,port=1` puts it on the root port.

The DHCP wait

With the card and the keyboard working, `!Boot` ran as far as `PreDesk` and stopped. It stayed stopped for three minutes, with the progress bar not moving.

PC samples put the machine in the `Internet`, `DHCP` and `SharedCLibrary` modules. The card image's Internet start-up file explained why. The RISC OS partition is FileCore, not FAT, so the file was found by searching the raw image for its text. It names `EtherUSB` as the Ethernet device, and runs the DHCP client with its *wait for the interface to appear* option on `ej0`.

That wait loop has three exits: the interface binds, the client gives up, or Escape is pressed. With no Ethernet-over-USB device there is no `ej0`, so it spins. This is stock RISC OS behaviour on a Pi with no network. The stock remedy is Escape, which takes the boot on to the desktop with a *machine startup has not completed successfully* box.

A network, without modelling GENET

Modelling the Pi 4's GENET Ethernet controller would have been the hardware answer. The soft answer was to supply the interface the boot sequence was waiting for, a different way.

RISC OS's `EtherUSB` driver has a generic **CDC Ethernet** backend. It needs a communications-class interface of subclass 6, a union descriptor naming the data interface, and an Ethernet functional descriptor carrying the MAC address. QEMU's `usb-net` is exactly that device in its CDC configuration, and QEMU's user-mode networking (slirp) behind it serves DHCP.

It did not bind the first time, and the trace said why: the device received only control transfers, never a bulk pipe. `usb-net` lists two configurations, **RNDIS first** and CDC second. RISC OS's USB driver selects the first configuration at enumeration, and passes on only that configuration's descriptors when it announces the device. So `EtherUSB` was shown an RNDIS interface with no Ethernet descriptor, and declined it.

Commit `7aebb77a43` adds an `rndis` property. It defaults to on, so nothing changes for anyone else. When it is off, the device offers only its CDC configuration, through the per-device descriptor override QEMU already has:

```
static void usb_net_realize(USBDevice *dev, Error **errp)
{
```

```

USBNetState *s = USB_NET(dev);

if (!s->offer_rndis) {
    dev->usb_desc = &desc_net_cdc;
}
usb_desc_create_serial(dev);
usb_desc_init(dev);
...
}

static const Property net_properties[] = {
    DEFINE_PROP_BOOL("rndis", USBNetState, offer_rndis, true),
    DEFINE_NIC_PROPERTIES(USBNetState, conf),
};

```

With `usb-net,rndis=off`:

- the configuration is set twice: once by the USB driver, and again by `EtherUSB` before it selects the data interface's alternate setting
- 44 frames go out on the bulk pipe during boot: the DHCP exchange and ARP
- there is **no DHCP wait at all**

Power-on to the desktop, idling in the `Portable` module with NetSurf open on the card image's Welcome page, took **27 seconds**. That was 19:57 on day one.

A pointer frozen in the corner

The next morning brought one more USB fix. The mouse worked, but a mouse reports relative motion, and a desktop inside a host window wants absolute positions. QEMU's `usb-tablet` provides those. Under RISC OS the tablet's pointer sat frozen in the corner of the screen.

RISC OS's USB mouse driver switches every HID device it claims to *report protocol* before opening its interrupt pipe, and treats a failure as fatal. QEMU accepted that request only from keyboards and mice, and stalled it for tablets. So the device was dropped. As commit `cc932f6a7f` notes, Linux and Windows never send `SET_PROTOCOL` to a device that describes its own reports, which is why nobody had seen this upstream. The tablet is a boot-capable HID device, so the request is legitimate:

```
case HID_SET_PROTOCOL:
    /* The tablet is a boot-capable HID device too (subclass 1,
     * protocol 2), and guests like RISC OS insist on switching it
     * to report protocol before they will open the interrupt pipe
     * -- stalling here makes them drop the device entirely. */
    if (hs->kind == HID_KEYBOARD || hs->kind == HID_MOUSE ||
        hs->kind == HID_TABLET) {
        hs->protocol = value;
    } else {
        goto fail;
    }
    break;
```

It was verified by injecting absolute events over QMP. An injected 8000,4000 out of 0..32767 reads back through RISC OS's MOUSE command as 390,1058, on a screen 1600 by 1200 OS units in size.

Snapshots, and the device that blocked them

One more network change came the next day. Upstream `usb-net` was marked unmigratable, and a single unmigratable device blocks `savevm` for the whole machine. Every standard RISC OS launch includes the network device.

Commit `bf68b11a71` gives it a real migration description, enough for the CDC path: the USB core state, the control-request results the guest cached, and any frame half-sent in either direction. The RNDIS response queue is not covered, because it is always empty in CDC mode.

The result is a `-loadvm` cold start with the desktop painted in **0.68 seconds**. The clock ticks, the caret blinks, the keyboard works, and a `*Ping` to the slirp gateway answers in 2 ms over the restored network.

The boot, measured

Sampling the PC every five seconds and naming the ROM module it was in gave the day-one boot profile:

Time	Where the machine is
0–7 s	ROM: the HAL and the kernel
12 s	Internet and DHCP: PreDesk finished
27 s	Portable: the desktop is up and the Wimp is idle

Without a network, the machine sits in `DHCP` from 12 seconds until Escape is pressed, and is on the desktop five seconds after that. Everything slower in earlier runs was the test harness waiting at fixed marks.

6. Time for an interrupt-driven OS

The day-one desktop worked, but it did not *keep time*. A centisecond ticker that should arrive a hundred times a second arrived 88.7 times, and one gap lasted 222 milliseconds. Windows dragged across the screen left half-drawn wreckage behind them. A restored snapshot came back with a dead clock.

This chapter covers the second day's work. It gave RISC OS its timing signals from a dedicated host thread, replaced two unimplemented timer blocks with thin latches, and fixed a DMA model that had been failing every window copy while reporting success.

What RISC OS waits for

RISC OS is interrupt-driven. Nothing in the desktop polls a clock; everything waits for a signal. On a Pi those signals come from three blocks in the SoC and from the GPU firmware.

The signals

Signal	Who waits for it	Source on a Pi	Rate
centisecond ticker	kernel: time, callbacks, Wimp	system timer compare 1	100 Hz
microsecond counter	kernel, between ticks	system timer counter	on demand
vertical sync	video driver, pointer, cursor	SMI interrupt, from firmware	per frame
half-frame update	video driver: palette, scroll	ARM timer, armed per vsync	per frame
USB start-of-frame	DWC driver's FIQ code	DWC2 frame interrupt	1 kHz

The vertical sync and the half-frame pulse are linked. At start-up the video driver claims the SMI interrupt and counts arrivals for 20 centiseconds. It keeps the real vsync **only if more than three arrive**. Otherwise it falls back to a fake vsync derived from the ticker, so a rate below about 20 Hz puts it on the fallback. Once on the real vsync, it uses the ARM timer's free-running counter to measure the frame and arms the timer for half of it. The half-frame pulse and the counter are both needed for its screen updates to keep flowing.

Upstream QEMU left both the ARM timer and the SMI block as unimplemented-device stubs.

A thread for the clock

QEMU's normal timers are serviced by the main loop. Under load the main loop gets round to them late. Over an idle desktop the ticker lost one tick in nine, with a worst gap of 222 ms.

Commit `d89e9bfb61` adds a new core facility, `system/hrtimer.c`: **one host thread, one list of deadlines, and one wait**. Its whole interface is three calls:

```
/* A timer whose callback runs on the timer thread, BQL held. */
HRTimer *hrtimer_new(HRTimerCB *cb, void *opaque);

/* Arm for an absolute QEMU_CLOCK_VIRTUAL deadline in ns; re-arming moves
 * it. Callable from any thread, including the callback itself. */
void hrtimer_mod_ns(HRTimer *t, int64_t deadline_ns);

/* Disarm. */
void hrtimer_del(HRTimer *t);
```

The thread

The thread loop is short, and its ordering is the important part:

```
for (;;) {
    now = qemu_clock_get_ns(QEMU_CLOCK_VIRTUAL);
    next = INT64_MAX;
    QTAILQ_FOREACH(t, &hr.armed, next) {
        next = MIN(next, t->deadline_ns);
    }
    if (next > now) {
        hrtimer_wait(MIN(next - now, (int64_t)HRTIMER_MAX_WAIT_NS));
```

```

        continue;
    }

    /* Something is due. Take the BQL first, then decide what fires,
     * so nothing re-arms behind our back between the two. */
    g_mutex_unlock(&hr.lock);
    bql_lock();
    g_mutex_lock(&hr.lock);
    now = qemu_clock_get_ns(QEMU_CLOCK_VIRTUAL);
    n = 0;
    QTAILQ_FOREACH_SAFE(t, &hr.armed, next, tnext) {
        if (t->deadline_ns <= now && n < HRTIMER_DUE_BATCH) {
            QTAILQ_REMOVE(&hr.armed, t, next);
            t->armed = false;
            due[n++] = t;
        }
    }
    g_mutex_unlock(&hr.lock);
    for (i = 0; i < n; i++) {
        due[i]->cb(due[i]->opaque);
    }
    bql_unlock();
    g_mutex_lock(&hr.lock);
}

```

Walking through it:

- **Find the earliest deadline** among the armed timers. If nothing is due, wait until it is, capped at 100 ms, so a stopped machine is re-checked now and then.
- **Something is due: take QEMU's big lock (the BQL) first, then decide what fires.** A guest write that re-arms a timer also runs under the BQL. Taking the lock *before* selecting the due timers means a re-arm cannot slip in between choosing a timer and firing it.
- **Re-read the clock under the lock**, take up to 16 due timers off the list, drop the list lock, and run their callbacks with the BQL held.

Deadlines are on QEMU's *virtual* clock. It runs with the host's monotonic clock while the machine runs, and stands still while the machine is stopped. The thread wakes at the host time a deadline would fall on. If the virtual clock has not got there, for example because the machine is paused, it simply waits again.

The wait itself

The wait is the one platform-specific piece:

```
#ifdef _WIN32
    due.QuadPart = -MAX(wait_ns / 100, (int64_t)1); /* relative, 100 ns */
    SetWaitableTimer(hr.timer, &due, 0, NULL, NULL, FALSE);
    g_mutex_unlock(&hr.lock);
    WaitForMultipleObjects(2, handles, FALSE, INFINITE);
    g_mutex_lock(&hr.lock);
#else
    g_cond_wait_until(&hr.cond, &hr.lock,
                    g_get_monotonic_time() + wait_ns / SCALE_US);
#endif
```

On Windows it is a *high-resolution waitable timer*. That takes its due time in 100 ns units and honours it to within tens of microseconds. On Windows versions before 10 1803, which lack that timer, the code falls back to a plain waitable timer and asks for a 1 ms scheduler period. Elsewhere it is a `GCond` wait with a microsecond deadline. On either path, a re-arm from another thread signals the wait so a nearer deadline is noticed at once. The thread also asks to be kept on the host's performance cores; chapter 7 measures that.

The system timer, rewired

The system timer's change is one line in the compare-register write. Each compare becomes a deadline on the thread instead of a main-loop timer:

```
triggers_delay_us = value - now;
trace_bcm2835_systmr_run(index, triggers_delay_us);
/* A deadline for the timer thread: RISC OS waits on this for its
 * 100 Hz ticker, so it fires when due, not at the next poll. */
```

```
hrtimer_mod_ns(s->tmr[index].timer,
              (now + triggers_delay_us) * SCALE_US);
```

Latches instead of timers

The ARM timer and the SMI block needed to exist, but not to be *modelled*. The fork replaces both unimplemented stubs with **latches**: registers that hold what the guest writes, plus an interrupt raised by something outside.

hw/timer/bcm2835_armtimer.c is the SP804-derived timer in the ARM control block, reduced to what the guest can observe. It has no countdown. Its interrupt is raised by a `fire` input, and only if the guest has enabled the timer:

```
static void bcm2835_armtimer_fire(void *opaque, int n, int level)
{
    BCM2835ARMTimerState *s = opaque;

    if (level && (s->control & CTRL_ENABLE)) {
        s->raw_irq = 1;
        s->fires++;
        bcm2835_armtimer_update_irq(s);
    }
}
```

The free-running counter the video driver uses to measure a frame is computed from the virtual clock on each read, at 1 MHz. No thread is needed for it.

hw/misc/bcm2835_smi.c is the vsync latch. The SMI block is really a parallel memory interface, but RISC OS uses it only for the firmware's vsync interrupt. So the model keeps just the control-and-status register. **Any write acknowledges**: RISC OS writes zero, and the datasheet's write-one-to-clear lands in the same place.

A generator for the frame

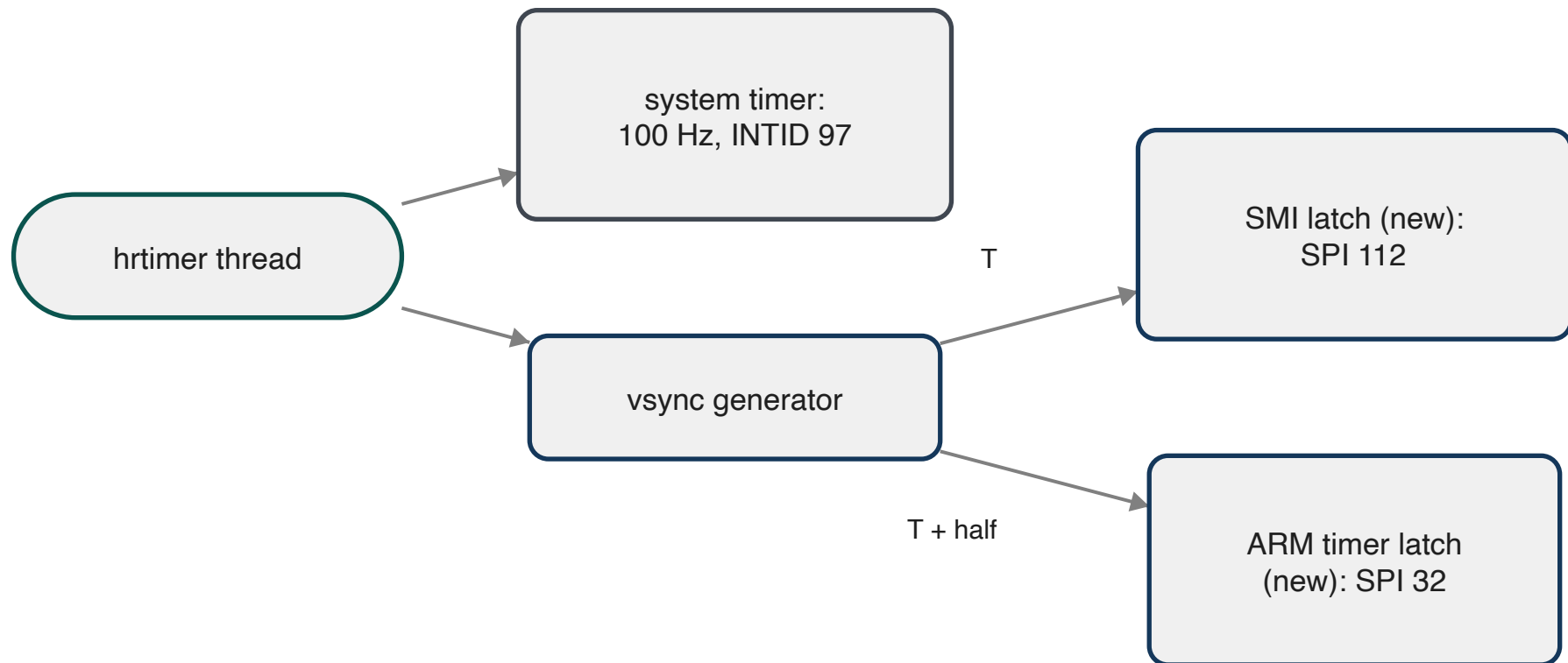
Something has to pulse those latches. `hw/misc/bcm2835_vsyncgen.c` runs on the timer thread with **two expiries per frame**: the vsync at time T , and the half-frame pulse at T plus half a period.

```
if (s->half_next) {
    qemu_irq_pulse(s->half_frame);
    s->half_next = false;
    hrtimer_mod_ns(s->timer, s->next_vsync_ns);
    return;
}
qemu_irq_pulse(s->vsync);
s->vsyncs++;
s->next_vsync_ns += period;
if (s->next_vsync_ns < now) {
    /* fell behind (machine stopped, host stalled): no catch-up burst */
    s->next_vsync_ns = now + period;
}
s->half_next = true;
hrtimer_mod_ns(s->timer, s->next_vsync_ns - period / 2);
```

Two design choices are visible here. **The schedule advances by whole periods**, so it does not drift. But **if it falls behind** — the machine was stopped, or the host stalled — it restarts from now instead of firing a burst of missed frames. A guest that is handed ten vsyncs at once gains nothing from them.

The default rate is **30 Hz**, set by the `hz` property. `-display dx11,vsync=N` or `-display metal,vsync=N` changes it, as does `-global bcm2835_vsyncgen.hz=N`. That sits comfortably above the 20 Hz below which the video driver would reject it. The host front end's render loop sends the guest nothing, so the guest's timing is independent of how the window is drawn.

One thread, three signals



Like the DWC2 line in chapter 5, the SMI and ARM timer interrupts are split so that they reach both the GIC and the legacy controller.

The result

	Before	After
ticks a second over an idle desktop	88.7	100.0
worst gap between ticks, Windows	222 ms	11.6 ms
worst gap between ticks, macOS	—	12.54 ms (p99 12.38 ms)

The macOS figure uses the thread's portable `GCond` branch. It was measured with a *second* full machine running on the same Mac. Replacing the wait with `mach_wait_until` on a real-time thread was considered and measured against first. The numbers showed nothing worth that machinery, so it was not built.

The video driver now adopts the real vsync, arms its half-frame interrupt, and its handler runs.

The clamp that was undone

One timing change was committed and then reversed within 18 minutes. The episode is a good example of the fork checking a fix against a trace before keeping it.

Commit `5a39b9150e` was work in progress saved as found in the tree, while a "desktop freeze" investigation changed hands. It included a clamp in the system timer: a compare value at or behind the counter would fire "on the next tick".

Commit `2a357a4a69` removed it, for three reasons:

1. **On silicon, the system timer matches on equality.** A compare value the counter has already passed is not reached until the 32-bit counter wraps. Guests that care handle this.
2. **RISC OS does handle it.** Its HAL rewrites the compare in a loop until the value is ahead of the counter. The trace showed exactly that: when a handler ran 40 ms late, four writes of "expire in 1 μ s" came first, then a sane one, and the tick never stopped.
3. **The freeze was not the timer at all.** The keyboard had been sending the guest the wrong keys. The clamp could also add a spurious tick, whenever its one-microsecond expiry beat the guest's next write.

The net change to upstream's compare semantics is **none**.

A snapshot with a dead clock

Snapshots exposed the thread's one blind spot. The compare registers migrate, but **armed deadlines do not**: they were in flight on the timer thread, and would have been in flight on an upstream QEMU timer too. After `-loadvm` every compare sat below the counter with no match possible. The 100 Hz ticker was dead, the guest spun on a clock that never moved, and typed keys were ignored.

The fix came as one of the "musts" in review commit `65c941595b`:

```
static int bcm2835_systmr_post_load(void *opaque, int version_id)
{
    BCM2835SystemTimerState *s = opaque;
    uint64_t now = qemu_clock_get_us(QEMU_CLOCK_VIRTUAL);

    for (int index = 0; index < BCM2835_SYSTIMER_COUNT; index++) {
        int32_t delay = (int32_t)(s->reg.compare[index] - (uint32_t)now);

        if (delay > 0) {
            hrtimer_mod_ns(s->tmr[index].timer,
                          (now + (uint64_t)delay) * SCALE_US);
        }
    }
    return 0;
}
```

The virtual clock is restored to its value at save time. So a compare that was armed for the future still is, and gets re-armed. One that is already due, or was never armed, stays quiet: the HAL has already corrected past it. **INFERRED**: upstream's version of the device has the same gap, as the code's own comment suggests.

The framebuffer device needed the same care in commit `6b90313543`. Its cached console mapping is not migrated and was never rebuilt after a restore, so the first screendump walked a zeroed mapping and crashed. Its console also kept a placeholder size. `post_load` now forces a redraw and resizes the

console. After both fixes a snapshot cold start painted the desktop in 0.95 s. The migratable network device in chapter 5 later brought that to 0.68 s.

The half-drawn desktop

The last day-two fix is a display bug that was really a DMA bug. After every window move or scroll, the desktop was left partly drawn.

RISC OS's kernel offers the video driver three render operations through `GraphicsV`: copy a rectangle, fill a rectangle, and wait until idle. On the Pi, the video driver implements only the copy, and it does it with the SoC's DMA controller in **2D mode**. It builds a control block with source and destination both incrementing, a **128-bit** transfer width, and a stride per row. For overlapping copies there are three cases:

The three copy cases

Case	How the copy runs
destination before source	forwards, from the top row
source before destination, rows not overlapping	from the bottom row upwards, with negative strides
rows overlapping	refused by the driver; the kernel copies in software

QEMU's DMA model **refused any transfer with the 128-bit width bit set**, and any length not a multiple of four bytes. It flagged an error and moved nothing. The video driver never checks for that error, so it reported the copy complete, and the Wimp redrew only the strips newly exposed. **That was the half-drawn desktop**. One drag of the NetSurf window by 120×140 pixels is about fourteen such copies of 621 pixels by up to 475, all backwards, on DMA channel 2.

Commit `7f4740ddc8` stops refusing, and moves each row whole when both ends increment:

```
/*
 * The width bits pick the bus transfer size on hardware; the bytes
 * moved are the same either way, so they are not a reason to
 * refuse, and neither is alignment: a transfer is whatever length
 * the guest asked for.
 */
```

```

bulk = (ch->ti & (BCM2708_DMA_S_INC | BCM2708_DMA_D_INC))
      == (BCM2708_DMA_S_INC | BCM2708_DMA_D_INC)
      && !(ch->ti & (BCM2708_DMA_S_IGNORE | BCM2708_DMA_D_IGNORE));

...
if (bulk && xlen) {
    /* both ends advance: move the row whole */
    dma_memory_read(&s->dma_as, ch->source_ad, row, xlen,
                    MEMTXATTRS_UNSPECIFIED);
    dma_memory_write(&s->dma_as, ch->dest_ad, row, xlen,
                     MEMTXATTRS_UNSPECIFIED);
    ch->source_ad += xlen;
    ch->dest_ad += xlen;
    xlen = 0;
}

```

The strides, including negative ones, are applied between rows as before. A new trace point, `bcm2835_dma_2d`, counts the 2D copies. A later commit from the blitter work, `64974a2686`, adds a fast path for pattern fills. **INFERRED inconsistency:** that fast path's comment says the video driver uses DMA fills for `FillRectangle`, but the design record says the driver does not implement `FillRectangle` at all.

The drawing side of the story goes much further. The kernel's software plotting was later taken onto the host by a paravirtual blitter. That is the subject of the [graphics and sound walkthrough](#).

7. Two hosts, and what is left

The fork was written on Windows: MSYS2 for the build, Direct3D 11 for the window, and a Win32 high-resolution waitable timer for the clock. On day two it moved to a Mac with Apple Silicon, and on day four to an Intel Mac. This chapter covers what moving it showed about the design, the measurements it made possible, how the work stands against QEMU's requirements for upstream contributions, and what is still open.

The Mac: the emulation was already portable

The macOS port record opens with a one-line verdict:

The emulation was already portable, and the front end was not.

Before any macOS code was written, an unmodified checkout was configured with QEMU's standard Cocoa display and built with Apple clang. It booted RISC OS 5.30 from the card to the 800×600 desktop, with NetSurf on the Welcome page, in under 20 seconds on an M4. The network worked from the first boot. The bare-metal channel-0 test printed `00000080`. The Python tools read the CMOS file out of the card image and built the blob unchanged.

What was platform-specific

Piece	Verdict on macOS
the VCHIQ and channel-0 peers, the vsync generator, the legacy interrupt controller, every device fix	portable; no <code>#ifdef _WIN32</code> anywhere in them
<code>riscos-pi4/tools/*.py</code>	portable
the Windows symlink workaround	already gated to Windows; a no-op
<code>system/hrtimer.c</code>	had a POSIX branch all along; measured in chapter 6 rather than assumed
<code>ui/dx11.{h,c,cpp}</code>	Windows only, and the whole of the work

Every RISC OS-specific change is plain C against QEMU's own interfaces. **So the port was not a port of the emulator. It was a new front end:**

`ui/metal.{h,c,m}`, the Metal twin of the Direct3D 11 window. Both front ends, and the paravirtual blitter behind them, are covered in the [graphics and sound walkthrough](#).

Speed on two hosts

Every macOS figure was measured with a *second* full machine running in a window on the same Mac, so the Mac numbers are a floor rather than a best case:

	i7-12700	M4	M4 against i7
power-on to a screen that stops changing	~27 s	20.8 s	1.3×
centisecond ticker, worst gap	11.6 ms	12.5 ms	—
bench: two instructions, registers only	~2,000 M/s	1,673 M/s	0.84×
bench2: six instructions, a load and a store	~500 M/s	895 M/s	1.8×

The split is the interesting part. A two-instruction register loop is almost pure TCG dispatch, and the i7's clock speed wins it. Add a load and a store, which is what real code does, and the M4 is nearly twice as fast. The boot involves device emulation, block I/O and the Wimp rather than a spin loop, and it follows the second number.

Two more figures place the emulator honestly:

- **The window costs nothing measurable.** Headless, a benchmark ran at about 1,580 M/s; with the window's full decode-and-present pipeline live at 60 Hz, about 1,540 M/s. That is -2.3% , within the $\pm 10\%$ noise of single samples. The 1.9 MB upload per frame runs on the UI thread and the GPU, off the vCPU's core.
- **Against real silicon**, the synthetic figures put a vCPU at *between a quarter and a half of one real Pi 4 core*, and worse on NEON-heavy code. That is TCG's usual standing, not a cost of this fork.

An Intel Mac

Commit `cf4a309fcd` took the fork to a Xeon W-3235 running macOS 26. Its problems were not in the emulator. Homebrew offered no prebuilt packages for several dependencies on that system, and refused to build them from source while an outdated Xcode sat at the default path.

The fork's answer was `tools/build-deps-macos.sh`. It builds `pcre2`, `glib`, `libslirp` and `capstone` statically, with the Command Line Tools alone, outside the repository, idempotently. Deleting or updating the user's Xcode was deliberately not made a build requirement.

Verified on that machine:

- the channel-0 regression prints `00000080`
- RISC OS boots from the card to a live, interactive desktop behind `-display metal`
- **the decode is pixel-faithful.** A capture from the Metal surface and a QMP screendump taken a second apart differ in **134 of 480,000 pixels**. They are a 12 × 22 block at dead centre: the pointer, mid-move between the two captures. Every other pixel is identical. The Windows acceptance test had the same shape: 84 pixels, the clock.

One method lesson is recorded so it is not relearned. An earlier comparison showed 89 % of pixels differing. It had compared a periodic capture of one desktop moment with a screendump of another. The periodic capture only fires on change, so the two images in a comparison must be taken close together.

Many cores: measured, then mostly left alone

A question came up on day three: can the emulator notice a host with fast and slow cores, and keep the busy vCPU on a fast one? It can notice. The measurements said forcing the choice was the wrong move.

Windows: pinning loses

Commit `e7699165ab` adds `qemu_thread_prefer_performance_cores()`, called from the TCG vCPU threads. On an i7-12700, with 8 performance and 4 efficiency cores, each run was one 30-second RISC OS boot. Retired guest instructions were counted with a TCG plugin, and the two arms were alternated so thermal drift fell on both equally:

Arm	Runs, billions of instructions	Mean
pinned to performance cores	2.55 / 3.15 / 2.54	2.75
left to the scheduler	3.14 / 3.19 / 3.20	3.18

Pinning is about 13 % slower, and much less even. A hard affinity mask overrides Intel's Thread Director, which already knows which core is fastest, and takes that choice away. An earlier single pair had suggested a 37 % *gain*. That was the first run of the session, on a cold chip, and alternating the arms is what exposed it.

So the default leaves placement to the scheduler, and pinning is available only through the `QEMU_VCPU_PIN` environment variable. By default the thread only opts out of Windows' efficiency quality-of-service class. That overrides nothing, and it measured as neutral (2.94 against 2.96 billion). It is kept for the case that could not be measured on a desktop: a laptop on battery, where that class is applied far more readily.

macOS: a QoS class, not a core

Commit `b46fa9765d` adds the macOS side. Apple Silicon offers no core pinning, but it has a quality-of-service ladder. A new pthread starts in the default class. It does **not** inherit its creator's class — that was measured, not assumed. The helper moves the vCPU threads, the timer thread and QEMU's main loop to `USER_INTERACTIVE`, which the scheduler keeps on performance cores while still choosing which one.

Quality of service on an M4

Measured with `bench2` on an M4 (4 performance and 6 efficiency cores), in M/s:

Performance cores also occupied by	Before	After
nothing	968–975	931–977
4 default-class busy loops	870	815
4 interactive-class busy loops	856	803
12 interactive-class busy loops	714	843

It is neutral until the machine is genuinely oversubscribed. Under oversubscription, one run without the change fell to 451 M/s, which is efficiency-core territory. With the change, no run fell below 764. That was one occurrence in three runs, so the record calls it a tendency rather than a rate.

The Xeon has twelve uniform cores, so the helper does nothing there. A darwin core-affinity arm could matter on a hybrid Intel Mac, but was not built. The fork's rule is measurement first.

A fix that fixed nothing

The fork's only change under `target/arm`, commit `1dd95655a9`, came from the HostFS work. The host was translating guest addresses and reporting some pages unmapped. That looked like a page-table-walk bug, so domain-fault checks were skipped for debug translations. The real cause was RISC OS mapping application space on demand: the pages genuinely were not mapped yet. The commit keeps the change, and says plainly that it **"demonstrably fixes nothing here, and can be dropped without loss."**

Upstreaming: the case for

The commit messages were written to be sent upstream. Most use QEMU's `subsystem:` subject style, explain the bug in guest-neutral terms, give the trace evidence, and name who else is affected. The fork identifies nine fixes as plain QEMU bugs.

Nine candidate fixes

Fix	Who else it affects
AArch32 secondary boot stub	any 32-bit guest on raspi3b or raspi4b
I2C byte access	any guest that uses STRB on the FIFO
I2C start on ST, with a TX FIFO	any guest following the documented sequence
system timer to the GIC	any guest using the system timer through the GIC on raspi4b
SD card on EMMC2	any raspi4b guest that uses the card

Nine candidate fixes, continued

Fix	Who else it affects
GIC legacy nFIQ inputs	any SoC wiring an older controller into a GIC-400's legacy inputs
DMA 2D transfers of any width	any guest using the DMA engine's 2D mode
DWC2 HCHINT and per-device IRQ level	any driver that defers a channel interrupt by masking it
usb-net rndis=off	any USB stack that announces a device in its first configuration

The plan for a give-back sprint adds three more to the series: the SD read-ahead, the channel-0 peer and the property tags. It keeps the VCHIQ peer, the EDID answer and the BCM2711 legacy controller in the fork. **INFERRED:** three more look generic too — the tablet `SET_PROTOCOL` fix, `usb-net` migration, and the timer and framebuffer `post_load` fixes.

Upstreaming: what stands in the way

As the code stands, several things would stop a submission. They fall into two groups. The AI co-author trailers break down as Claude Opus 5 on 82 commits, Claude Fable 5.1 on 31, ZCode GLM on 5 and Claude Opus 4.8 on 4.

Process and policy

Issue	Detail
QEMU's policy on AI-generated content	docs/devel/code-provenance.rst declines contributions believed to include AI-generated content; 122 of the 156 commits carry AI co-author trailers
No sign-offs	no commit carries a Signed-off-by line, which QEMU requires under its Developer's Certificate of Origin
No tests	nothing under tests/ changed; the channel-0 check is a bare-metal blob outside QEMU's test harness
Not for upstream by its own label	the Windows symlink workaround says so in its commit message

The shape of the code

Issue	Detail
Blast radius (INFERRED)	the channel-0 peer, VCHIQ, ARM timer, SMI latch and 30 Hz vsync generator are created in the code shared by every raspi machine; only RISC OS on raspi4b is reported tested
A parallel timer system (INFERRED, untested)	system/hrtimer.c sits outside QEMU's timer lists, so -icount and record/replay are likely not honoured
Layering (INFERRED)	a guest-visible rate, the vsync, is set through a -display option; the front ends find the framebuffer by object path

The shape of the code, continued

Issue	Detail
Migration compatibility (INFERRED)	bcm2835_i2c and bcm2835_property bumped their version from 1 to 2 instead of adding subsections
Missing licence notices	hw/misc/vmchannel.c, include/hw/misc/vmchannel.h, include/hw/misc/riscos_blitter.h, ui/dx11.c, ui/dx11.cpp, ui/dx11.h and ui/metal.h
Documentation	docs/system/arm/raspi.rst is untouched; the rndis and mode properties are undocumented
A plan not built	the design called for a separate raspi4b32 machine name; it does not exist in the code

What is left

Open limitations

- **SET_CLOCK_RATE** is still not implemented; it only logs.
- **The card stall.** About a millisecond per block during boot is measured but not understood.
- **GENET and the VL805** USB 3 controller behind PCIe are not modelled. EtherGENET still needs the CMOS unplug bit or the patched ROM.
- **The USB start-of-frame FIQ** runs at 1 kHz from QEMU's main loop. It is the next candidate for the timer thread.
- **EDID** stops at 1920×1200, with no CTA-861 extension block. The Direct3D 11 decoder also has a texture-width ceiling of 16,384.
- **Later sprints not started:** `roscc` on the real target, the debugger, benchmarks against hardware, record and replay, the upstream series, and packaging.
- **macOS:** pointer grab, full screen and the snapshot menu are unexercised, and moving between displays of different DPI is untested. The Apple Events scripting interface is built through stage E3; breakpoints, single step and signing, planned as E4 to E6, are not built.

Documents that have fallen behind

- `DESIGN.md` still gives its status as "Sprint 2 done". Its blocker table still lists the abandoned EDID EEPROM.
- The fork banner in `README.rst` still says keyboard input is not proven.
- The VCHIQ peer's header comment, and a README passage, still describe a small device that refuses every service.
- The README cites a design-record section for the card stall that does not contain it.
- The DMA fill comment disagrees with the design record about FillRectangle.

Risks found by reading, not by testing

- **A seqlock reader that can return nothing.** `bcm2835_fb_get_config` retries while the generation is odd. But the `continue` inside its `do ... while` jumps to the loop condition. If a write is in progress and the generation has not moved between the two reads, the loop exits without writing `*out`. The callers in both front ends pass an uninitialised stack structure. The window is tiny.
- **Record and replay.** Timer-thread deadlines are likely invisible to `-icount` and replay.
- **Other guests.** The new common-code devices could change the behaviour of other raspi guests, which nobody has tested.

The fork at a glance

Change	Commit
AArch32 secondary stub, chosen by CPU state	86a43faa32
mailbox channel-0 power peer	4f3f926d0c
system timer to the GIC	57651d54ce
VCHIQ peer; bounds checks	1013487c14, baddb065d3
PCIe and GENET stubs	1a22c9b0b1, 418847ac82, 5a61a7881c
SD card on EMMC2; 64 KiB read-ahead	a7ea3f48da, e48ee2c73a

The fork at a glance, continued: tags, I2C and interrupts

Change	Commit
property tags: buffers, GPIO state	f87fdd5f72, 42de159040
EDID: 800×600, then wide modes	70522977c7, 7ef62fa51f
I2C: byte access, sub-word access, ST start and TX FIFO	d742c3b617, 57724dbd8e, 59e0d5b5ea
BCM2711 legacy controller; GIC legacy FIQ bypass	873dccc24c, 54eac9c9fb
DWC2 HCHINT and IRQ level	622a7bde8c

The fork at a glance, continued: USB, time, DMA and hosts

Change	Commit
rndis=off; usb-net migration; tablet protocol	7aebb77a43, bf68b11a71, cc932f6a7f
timer thread, ARM timer and SMI latches, vsync generator	d89e9bfb61
compare clamp added, then undone; post_load re-arm	5a39b9150e, 2a357a4a69, 65c941595b
DMA 2D transfers of any width, rows whole	7f4740ddc8
hybrid-core placement, Windows then macOS	e7699165ab, b46fa9765d
debug-walk domain check, self-declared unnecessary	1dd95655a9